FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

3

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Student Name	SANGKULA, Chedrick Khylle P. / SAGPANG, Edriane
Section:	[Section]
Professor:	[Professor]
Date:	November 3, 2026

I. BACKGROUND INFORMATION

Case-Based Reasoning (CBR) is an intelligent problem-solving approach used in **Knowledge Representation and Reasoning (KRR)**. Instead of relying solely on predefined rules, CBR systems solve new problems by referring to past experiences, also known as cases. These cases contain previous problems and their successful solutions. When a new situation arises, the system identifies the most similar past case, adapts the old solution to fit the new context, and applies it. Afterward, the new case is saved, allowing the system to learn and continuously improve.

CBR is widely used in various fields, such as disaster response, legal assistance, and intelligent tutoring, because it mimics how humans naturally make decisions. For example, emergency responders rely on past typhoon data to plan evacuations; lawyers examine previous legal cases to build arguments; and tutoring systems look at prior student mistakes to provide customized guidance. In all these examples, past knowledge becomes the foundation for solving future problems more effectively.

This activity allows students to experience how CBR works by developing a simple, functional CBR program based on real-world scenarios. Through pair programming, students will learn how to represent cases, implement similarity assessment, adapt solutions, and store new learning experiences—reflecting the complete CBR cycle. This hands-on approach strengthens understanding of how intelligent systems use experiential knowledge to support reasoning, decision-making, and learning.

II. TECHNICAL ASSESSMENT

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Objective

At the end of this laboratory activity, students are expected to:

1. **Understand and apply the Case-Based Reasoning (CBR) cycle**, including retrieval, reuse, revision, and retention.
2. **Represent cases effectively** using appropriate data structures such as dictionaries, arrays, JSON objects, or classes.
3. **Implement a similarity assessment method** to identify relevant past cases based on defined attributes.
4. **Adapt retrieved solutions** to meet the needs of a new problem scenario based on the chosen case study.
5. **Demonstrate learning in CBR** by saving the newly solved case into the system's case base.
6. **Develop collaboration skills** through pair programming and joint problem analysis.
7. **Apply knowledge representation and reasoning concepts** to real-world problems in disaster response, legal reasoning, or intelligent tutoring.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)

2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

4. Reference Materials

Lecture notes on:

- Knowledge Representation & Reasoning
- Case-Based Reasoning cycle
- Similarity measures

Provided case studies (Disaster DSS, Legal DSS, Intelligent Tutoring System)

5. Pair Collaboration Tools (optional but useful)
    - Google Drive for shared documentation
    - GitHub/GitLab for code collaboration

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Please ensure that you are grouped as pairs in the People tab.
3. Select only one case study from the list below.
4. Include clear comments in your code to describe its logic.
5. Each program should display the correct input, process, and output.
6. Ensure that your script displays clear and readable outputs.
7. Compile all Python codes and corresponding outputs into one PDF file.
8. Save it as **TA3_lastname1_lastname2.pdf**
   (Example: TA2_Patilano_Loverez.pdf)
9. Submit the **python file** and **PDF** file.

📌 Step-by-Step Instructions

1. Form a pair

Choose a partner. Each pair will work collaboratively on research, coding, debugging, and presenting your results.

2. Choose ONE (1) case study

Select only one case study from the list below. Your program must be based on the scenario and logic of the chosen system:

Case Study Options:

1. *Disaster Response Decision Support System (DSS)*
2. *AI-Based Legal Decision Support System*
3. *Intelligent Tutoring System (ITS)*

3. Define your case base

Create at least **3–5 past cases** stored as dictionaries, JSON objects, arrays, or any structure.

Example structure:

```
{
  "problem": {...},
```

```
  "solution": {...}
}
```

4. Implement similarity assessment

Your program must include a way to measure similarity between the new case and past cases. Examples:

- Attribute matching
- Weighted scoring
- Simple rule-based comparison

Output should show: (sample)
✔️ Similarity score or ranking
✔️ The most relevant past case retrieved

5. Adapt the solution

Modify the retrieved solution so it fits the new problem scenario.

This part should reflect the logic of your chosen case study:

Examples:
• **DSS:** Adjust evacuation plans based on population size
• **Legal DSS:** Modify arguments due to new digital evidence
• **ITS:** Give personalized feedback based on student errors

6. Demonstrate learning (retain stage)

After generating a final solution, store the new case into your case base.

Your program must show:
✔️ "New case saved" or similar confirmation
✔️ Updated case list

7. Display clear program outputs

Your output must include:

- New problem description
- Retrieved similar case
- Adapted final solution
- Updated case base

8. Submission Requirements

Submit the following:

A. Program File

- Python

B. Short Documentation (1–2 pages)

Include:

- Title of chosen case study
- Description of your case base
- Explanation of similarity method
- Adaptation logic
- Screenshots of program execution

C. Pair Reflection (5–7 sentences)

Discuss:

- What each partner contributed
- What challenges you encountered
- What you learned about CBR

Case Studies:

Case Study 1: Disaster Response Decision Support System

A CBR-based DSS helps local government units respond to natural disasters.

- Past cases include typhoons, floods, and earthquakes.
- New event: *Typhoon with expected flooding in urban areas.*
- System retrieves similar events: *Typhoon Ursula (2019)* and *Typhoon Odette (2021)*.
- It adapts recommended responses such as evacuation strategy, resource allocation, and shelter capacity.
- After execution, the system stores new outcomes for improved future reasoning.

Learning Outcome:

CBR helps governments respond faster by using *experiential knowledge* rather than rebuilding decisions from scratch.

Case Study 2: AI-Based Legal Decision Support System

A legal CBR tool assists lawyers by retrieving previous court cases similar to the current legal issue.

Scenario:

Client is charged with “theft involving electronic transfer.” System retrieves past cases involving:

- online theft
- digital fraud
- unauthorized bank transfers

Adaptation:

The reasoning engine adjusts argument patterns to match the new technical context.

Learning:

After final verdict, the case is stored—enhancing the system's knowledge for future cybercrime-related offenses.

Case Study 3: Intelligent Tutoring System
A CBR system identifies the learning struggles of students. Student submits a math solution with repeated errors. System retrieves past students with similar error patterns (e.g., misusing formulas). It adapts feedback and personalized lessons based on successful interventions. **Learning:** System improves by retaining new student profiles and outcomes.

Your Code - Laboratory Tasks ⇩

A. Program File (cbr_disaster_dss.py)

The program below is written in Python 3 and uses only built-in modules (json, math, os, sys, datetime). Run it with python cbr_disaster_dss.py for the demo scenario, python cbr_disaster_dss.py --interactive to type a new disaster, or python cbr_disaster_dss.py --reset to restore the original five cases. The case base is saved to case_base.json so that learned cases persist between runs.

```
"""
TA3 - Case-Based Reasoning (CBR) System Development
Case Study 1: Disaster Response Decision Support System (DSS)

A simple CBR program that helps a Local Government Unit (LGU) plan its
response to a new disaster by reusing experience from past disasters.

The program follows the full CBR cycle (the "4 R's"):
    1. RETRIEVE - find the most similar past disaster(s) in the case base
    2. REUSE    - take the solution of the best-matching case
    3. REVISE   - adapt that solution to fit the new disaster
    4. RETAIN   - save the new problem + adapted solution as a new case

Run:
    python cbr_disaster_dss.py              -> runs the demo scenario
    python cbr_disaster_dss.py --interactive -> lets you type a new event
    python cbr_disaster_dss.py --reset       -> restores the original case base
```

```
Note: Case values are simplified, sample data for a class activity,
loosely inspired by real Philippine events.
"""

import json
import math
import os
import sys
from datetime import datetime

# File where the case base is stored (so learning persists between runs)
CASE_BASE_FILE = "case_base.json"


# ============================================================================
# 1. CASE REPRESENTATION
# Each case is a dictionary with three parts:
#   - "id" / "name"  : identifies the case
#   - "problem"      : features that describe the disaster situation
#   - "solution"     : the response plan that was used
#   - "outcome"      : how well the plan worked (used for learning)
# ============================================================================
INITIAL_CASES = [
    {
        "id": 1,
        "name": "Typhoon Ursula (2019)",
        "problem": {
            "disaster_type": "typhoon",
            "area_type": "rural",
            "wind_speed_kph": 195,
            "rainfall_mm": 150,
            "flood_risk": "medium",
            "affected_population": 120000,
            "warning_lead_time_hrs": 48,
        },
        "solution": {
            "evacuation_strategy": "Pre-emptive evacuation of coastal barangays",
            "evacuees": 30000,
            "evacuation_centers": 60,
            "shelter_capacity": 33000,
            "food_packs": 90000,
            "rescue_teams": 25,
            "rescue_boats": 10,
            "medical_teams": 12,
        },
        "outcome": {"success_rating": 4, "notes": "Coastal areas cleared early."},
    },
    {
        "id": 2,
        "name": "Typhoon Odette (2021)",
        "problem": {
            "disaster_type": "typhoon",
            "area_type": "urban",
            "wind_speed_kph": 195,
            "rainfall_mm": 250,
            "flood_risk": "high",
            "affected_population": 300000,
            "warning_lead_time_hrs": 36,
        },
        "solution": {
            "evacuation_strategy": "Forced evacuation of low-lying and coastal zones; schools used as
shelters",
            "evacuees": 90000,
            "evacuation_centers": 150,
            "shelter_capacity": 99000,
            "food_packs": 270000,
            "rescue_teams": 60,
```

```
            "rescue_boats": 35,
            "medical_teams": 30,
        },
        "outcome": {"success_rating": 3, "notes": "Shelters overcrowded on day 2."},
    },
    {
        "id": 3,
        "name": "Tropical Storm Ondoy (2009)",
        "problem": {
            "disaster_type": "flood",
            "area_type": "urban",
            "wind_speed_kph": 85,
            "rainfall_mm": 450,
            "flood_risk": "high",
            "affected_population": 500000,
            "warning_lead_time_hrs": 12,
        },
        "solution": {
            "evacuation_strategy": "Vertical evacuation to upper floors + boat rescue in flooded
streets",
            "evacuees": 150000,
            "evacuation_centers": 220,
            "shelter_capacity": 165000,
            "food_packs": 450000,
            "rescue_teams": 120,
            "rescue_boats": 80,
            "medical_teams": 45,
        },
        "outcome": {"success_rating": 2, "notes": "Short warning time; not enough boats."},
    },
    {
        "id": 4,
        "name": "Typhoon Ulysses (2020)",
        "problem": {
            "disaster_type": "typhoon",
            "area_type": "urban",
            "wind_speed_kph": 155,
            "rainfall_mm": 350,
            "flood_risk": "high",
            "affected_population": 400000,
            "warning_lead_time_hrs": 24,
        },
        "solution": {
            "evacuation_strategy": "Staged evacuation of riverside communities; roof-top rescue",
            "evacuees": 110000,
            "evacuation_centers": 180,
            "shelter_capacity": 121000,
            "food_packs": 330000,
            "rescue_teams": 90,
            "rescue_boats": 60,
            "medical_teams": 35,
        },
        "outcome": {"success_rating": 3, "notes": "Dam release worsened flooding."},
    },
    {
        "id": 5,
        "name": "Bohol Earthquake (2013)",
        "problem": {
            "disaster_type": "earthquake",
            "area_type": "rural",
            "wind_speed_kph": 0,
            "rainfall_mm": 0,
            "flood_risk": "low",
            "affected_population": 200000,
            "warning_lead_time_hrs": 0,
        },
        "solution": {
```

```
            "evacuation_strategy": "Move residents to open fields; structural checks before return",
            "evacuees": 50000,
            "evacuation_centers": 80,
            "shelter_capacity": 55000,
            "food_packs": 150000,
            "rescue_teams": 40,
            "rescue_boats": 0,
            "medical_teams": 25,
        },
        "outcome": {"success_rating": 4, "notes": "Tent cities set up quickly."},
    },
]


# ============================================================================
# Case base helpers (load / save)
# ============================================================================
def load_case_base():
    """Load cases from the JSON file; create it from INITIAL_CASES if missing."""
    if not os.path.exists(CASE_BASE_FILE):
        save_case_base(INITIAL_CASES)
    with open(CASE_BASE_FILE, "r", encoding="utf-8") as f:
        return json.load(f)


def save_case_base(cases):
    """Write the whole case base back to the JSON file."""
    with open(CASE_BASE_FILE, "w", encoding="utf-8") as f:
        json.dump(cases, f, indent=2)


# ============================================================================
# 2. SIMILARITY ASSESSMENT
# Weighted (global) similarity = sum(weight * local_similarity) / sum(weights)
#   - Categorical features: exact match = 1, related = partial, else 0
#   - Numeric features: 1 - |a - b| / range   (closer values -> higher score)
# ============================================================================

# How important each feature is when comparing disasters (total = 10)
WEIGHTS = {
    "disaster_type": 2.5,
    "area_type": 1.5,
    "flood_risk": 2.0,
    "rainfall_mm": 1.0,
    "wind_speed_kph": 1.0,
    "affected_population": 1.5,
    "warning_lead_time_hrs": 0.5,
}

# Value range of numeric features (used to normalize differences to 0..1)
NUMERIC_RANGES = {
    "rainfall_mm": 500,
    "wind_speed_kph": 300,
    "affected_population": 1_000_000,
    "warning_lead_time_hrs": 72,
}

# Partial similarity between related disaster types
# (a typhoon often causes flooding, so they are 60% similar)
TYPE_SIMILARITY = {
    ("typhoon", "flood"): 0.6,
    ("flood", "typhoon"): 0.6,
}

# Ordinal levels for flood risk (low < medium < high)
RISK_LEVEL = {"low": 0, "medium": 1, "high": 2}
```

```
def local_similarity(feature, new_value, old_value):
    """Similarity (0..1) between two values of ONE feature."""
    if feature == "disaster_type":
        if new_value == old_value:
            return 1.0
        return TYPE_SIMILARITY.get((new_value, old_value), 0.0)

    if feature == "area_type":
        return 1.0 if new_value == old_value else 0.0

    if feature == "flood_risk":
        # one level apart = 0.5, two levels apart = 0
        diff = abs(RISK_LEVEL[new_value] - RISK_LEVEL[old_value])
        return 1.0 - diff / 2

    # Numeric features
    rng = NUMERIC_RANGES[feature]
    diff = abs(new_value - old_value)
    return max(0.0, 1.0 - diff / rng)


def compute_similarity(new_problem, old_problem):
    """Global weighted similarity (0..1) between a new and a past problem."""
    total, weight_sum = 0.0, 0.0
    for feature, weight in WEIGHTS.items():
        sim = local_similarity(feature, new_problem[feature], old_problem[feature])
        total += weight * sim
        weight_sum += weight
    return total / weight_sum


def retrieve(new_problem, cases, k=3):
    """RETRIEVE: rank all past cases by similarity and return the top k."""
    ranked = []
    for case in cases:
        score = compute_similarity(new_problem, case["problem"])
        ranked.append((score, case))
    ranked.sort(key=lambda x: x[0], reverse=True)
    return ranked[:k], ranked


# ======================================================================
# 3. CASE ADAPTATION (REUSE + REVISE)
# The retrieved plan is adjusted to the new event using simple rules:
#   - Scale evacuees and resources by the population ratio
#   - Increase evacuation rate if flood risk / rainfall is higher
#   - Add more rescue boats for urban flooding
#   - Add a 10% shelter buffer (more if the old case had overcrowding)
#   - Change evacuation strategy if warning time is short
# ======================================================================
def round_up(x):
    """Round up to a whole number (round first to ignore tiny float errors)."""
    return math.ceil(round(x, 6))


def adapt_solution(new_problem, retrieved_case):
    old_p = retrieved_case["problem"]
    old_s = retrieved_case["solution"]
    notes = []  # explanation of each adaptation (for transparency)

    # --- Rule 1: scale by affected population ---
    pop_ratio = new_problem["affected_population"] / old_p["affected_population"]
    notes.append(f"Population ratio = {new_problem['affected_population']:,} / "
                 f"{old_p['affected_population']:,} = {pop_ratio:.2f}")

    # --- Rule 2: evacuation rate depends on flood risk and rainfall ---
```

```
    old_rate = old_s["evacuees"] / old_p["affected_population"]
    risk_factor = {"low": 0.8, "medium": 1.0, "high": 1.2}[new_problem["flood_risk"]] / \
                  {"low": 0.8, "medium": 1.0, "high": 1.2}[old_p["flood_risk"]]
    rain_factor = 1.0
    if new_problem["rainfall_mm"] > old_p["rainfall_mm"]:
        rain_factor = 1.10
        notes.append("Rainfall is higher than the past case -> evacuate 10% more people")
    new_rate = min(old_rate * risk_factor * rain_factor, 0.60)  # cap at 60%
    evacuees = round_up(new_problem["affected_population"] * new_rate)
    notes.append(f"Evacuation rate {old_rate:.1%} -> {new_rate:.1%}  =>  {evacuees:,} evacuees")

    # --- Rule 3: shelters (10% buffer, 20% if the past case was overcrowded) ---
    buffer = 1.20 if "overcrowd" in retrieved_case["outcome"]["notes"].lower() else 1.10
    if buffer > 1.10:
        notes.append("Past case reported overcrowding -> shelter buffer raised to 20%")
    shelter_capacity = round_up(evacuees * buffer)
    people_per_center = old_s["shelter_capacity"] / old_s["evacuation_centers"]
    evacuation_centers = round_up(shelter_capacity / people_per_center)

    # --- Rule 4: relief and response resources scale with evacuees ---
    evac_ratio = evacuees / old_s["evacuees"]
    food_packs = round_up(evacuees * 3)  # 3 days of food packs per evacuee
    rescue_teams = round_up(old_s["rescue_teams"] * evac_ratio)
    medical_teams = round_up(old_s["medical_teams"] * evac_ratio)
    rescue_boats = round_up(old_s["rescue_boats"] * evac_ratio)

    # --- Rule 5: urban flooding needs extra boats ---
    if new_problem["area_type"] == "urban" and new_problem["flood_risk"] == "high":
        rescue_boats = round_up(rescue_boats * 1.25)
        notes.append("Urban area with HIGH flood risk -> +25% rescue boats")

    # --- Rule 6: strategy depends on warning lead time ---
    strategy = old_s["evacuation_strategy"]
    if new_problem["warning_lead_time_hrs"] < 24:
        strategy = "IMMEDIATE forced + vertical evacuation; " + strategy
        notes.append("Warning time < 24 hrs -> switch to immediate/forced evacuation")
    elif new_problem["warning_lead_time_hrs"] >= 48:
        strategy = "Staged departures over the 48-hr window; " + strategy
        notes.append("Warning time >= 48 hrs -> staged pre-emptive evacuation")

    adapted = {
        "evacuation_strategy": strategy,
        "evacuees": evacuees,
        "evacuation_centers": evacuation_centers,
        "shelter_capacity": shelter_capacity,
        "food_packs": food_packs,
        "rescue_teams": rescue_teams,
        "rescue_boats": rescue_boats,
        "medical_teams": medical_teams,
    }
    return adapted, notes


def revise_solution(solution):
    """REVISE: simple sanity checks so the plan is safe before it is used."""
    checks = []
    if solution["shelter_capacity"] < solution["evacuees"]:
        solution["shelter_capacity"] = round_up(solution["evacuees"] * 1.1)
        checks.append("Shelter capacity raised to cover all evacuees")
    if solution["medical_teams"] < 1:
        solution["medical_teams"] = 1
        checks.append("At least 1 medical team assigned")
    if not checks:
        checks.append("All checks passed (shelter >= evacuees, resources > 0)")
    return solution, checks
```

```
# ============================================================================
# 4. LEARNING (RETAIN)
# The new problem + adapted solution is stored as a new case
# so the system can reuse it next time.
# ============================================================================
def retain(cases, name, new_problem, solution, based_on, rating=4):
    new_case = {
        "id": max(c["id"] for c in cases) + 1,
        "name": name,
        "problem": new_problem,
        "solution": solution,
        "outcome": {
            "success_rating": rating,
            "notes": f"Adapted from '{based_on}' on {datetime.now():%Y-%m-%d %H:%M}",
        },
    }
    cases.append(new_case)
    save_case_base(cases)
    return new_case


# ============================================================================
# Display helpers
# ============================================================================
def line(char="=", n=78):
    print(char * n)


def show_problem(p):
    print(f"  Disaster type        : {p['disaster_type']}")
    print(f"  Area type            : {p['area_type']}")
    print(f"  Wind speed           : {p['wind_speed_kph']} kph")
    print(f"  Rainfall             : {p['rainfall_mm']} mm")
    print(f"  Flood risk           : {p['flood_risk']}")
    print(f"  Affected population  : {p['affected_population']:,}")
    print(f"  Warning lead time    : {p['warning_lead_time_hrs']} hrs")


def show_solution(s):
    print(f"  Evacuation strategy  : {s['evacuation_strategy']}")
    print(f"  Evacuees             : {s['evacuees']:,}")
    print(f"  Evacuation centers   : {s['evacuation_centers']:,}")
    print(f"  Shelter capacity     : {s['shelter_capacity']:,}")
    print(f"  Food packs           : {s['food_packs']:,}")
    print(f"  Rescue teams         : {s['rescue_teams']:,}")
    print(f"  Rescue boats         : {s['rescue_boats']:,}")
    print(f"  Medical teams        : {s['medical_teams']:,}")


def show_case_base(cases):
    print(f"  {'ID':<4}{'Case name':<36}{'Type':<12}{'Area':<7}{'Flood':<8}{'Population':>11}")
    line("-")
    for c in cases:
        p = c["problem"]
        print(f"  {c['id']:<4}{c['name'][:35]:<36}{p['disaster_type']:<12}"
              f"{p['area_type']:<7}{p['flood_risk']:<8}{p['affected_population']:>11,}")


# ============================================================================
# Input of a new problem
# ============================================================================
def demo_problem():
    """New event from the case study: typhoon with expected urban flooding."""
    return "New Typhoon - Urban Flooding", {
        "disaster_type": "typhoon",
        "area_type": "urban",
        "wind_speed_kph": 175,
```

```
        "rainfall_mm": 320,
        "flood_risk": "high",
        "affected_population": 350000,
        "warning_lead_time_hrs": 18,
    }


def ask(prompt, options=None, cast=str):
    """Ask until a valid value is typed."""
    while True:
        value = input(prompt).strip().lower()
        try:
            value = cast(value)
        except ValueError:
            print("   Please enter a number.")
            continue
        if options and value not in options:
            print(f"   Choose one of: {', '.join(options)}")
            continue
        return value


def input_problem():
    print("\nEnter the details of the NEW disaster event:")
    name = input("  Event name: ").strip() or "Unnamed Event"
    problem = {
        "disaster_type": ask("  Disaster type (typhoon/flood/earthquake): ",
                             ["typhoon", "flood", "earthquake"]),
        "area_type": ask("  Area type (urban/rural): ", ["urban", "rural"]),
        "wind_speed_kph": ask("  Wind speed (kph): ", cast=int),
        "rainfall_mm": ask("  Expected rainfall (mm): ", cast=int),
        "flood_risk": ask("  Flood risk (low/medium/high): ", ["low", "medium", "high"]),
        "affected_population": ask("  Affected population: ", cast=int),
        "warning_lead_time_hrs": ask("  Warning lead time (hrs): ", cast=int),
    }
    return name, problem


# ============================================================
# MAIN - runs the full CBR cycle
# ============================================================
def main():
    if "--reset" in sys.argv:
        save_case_base(INITIAL_CASES)
        print("Case base reset to the 5 original cases.")
        return

    line()
    print("   CASE-BASED REASONING: DISASTER RESPONSE DECISION SUPPORT SYSTEM")
    line()

    cases = load_case_base()
    print(f"\nCase base loaded: {len(cases)} past cases")
    show_case_base(cases)

    # ---------- INPUT: new problem ----------
    if "--interactive" in sys.argv:
        name, new_problem = input_problem()
    else:
        name, new_problem = demo_problem()

    print()
    line()
    print(f" [INPUT] NEW PROBLEM: {name}")
    line()
    show_problem(new_problem)
```

```
    # ---------- STEP 1: RETRIEVE ----------
    top_k, ranked = retrieve(new_problem, cases, k=3)
    print()
    line()
    print(" [STEP 1] RETRIEVE - Similarity ranking of all past cases")
    line()
    for rank, (score, case) in enumerate(ranked, start=1):
        bar = "#" * int(score * 30)
        print(f"  {rank}. {case['name'][:35]:<36}{score:7.2%}  {bar}")

    best_score, best_case = top_k[0]
    print(f"\n  >> Most similar case: {best_case['name']} (similarity = {best_score:.2%})")
    print(f"  >> Other similar cases: " +
          ", ".join(f"{c['name']} ({s:.0%})" for s, c in top_k[1:]))

    # Feature-by-feature breakdown for the best case (explains the score)
    print("\n  Feature breakdown vs. best case:")
    print(f"  {'Feature':<24}{'New':>12}{'Past':>12}{'Local sim':>11}{'Weight':>8}")
    for f, w in WEIGHTS.items():
        nv, ov = new_problem[f], best_case["problem"][f]
        print(f"  {f:<24}{str(nv):>12}{str(ov):>12}"
              f"{local_similarity(f, nv, ov):>11.2f}{w:>8.1f}")

    # ---------- STEP 2: REUSE ----------
    print()
    line()
    print(f" [STEP 2] REUSE - Solution of '{best_case['name']}'")
    line()
    show_solution(best_case["solution"])
    print(f"  Past outcome         : {best_case['outcome']['success_rating']}/5 - "
          f"{best_case['outcome']['notes']}")

    # ---------- STEP 3: REVISE (adapt) ----------
    adapted, notes = adapt_solution(new_problem, best_case)
    adapted, checks = revise_solution(adapted)
    print()
    line()
    print(" [STEP 3] REVISE - Adaptation rules applied")
    line()
    for n in notes:
        print(f"  * {n}")
    for c in checks:
        print(f"  * Check: {c}")

    print()
    line()
    print(" [OUTPUT] ADAPTED FINAL SOLUTION")
    line()
    show_solution(adapted)

    # ---------- STEP 4: RETAIN ----------
    new_case = retain(cases, name, new_problem, adapted, best_case["name"])
    print()
    line()
    print(" [STEP 4] RETAIN - Learning")
    line()
    print(f"  New case saved! (Case ID {new_case['id']}: {new_case['name']})")
    print(f"  Case base now has {len(cases)} cases (saved to {CASE_BASE_FILE})\n")
    print("  UPDATED CASE BASE:")
    show_case_base(cases)
    line()


if __name__ == "__main__":
    main()
```

B. Short Documentation

Title of Chosen Case Study

Case Study 1: Disaster Response Decision Support System (DSS) — a CBR program that helps a local government unit plan its evacuation strategy, shelter capacity, and resource allocation for a new disaster by reusing the response plans of similar past disasters.

Description of the Case Base

The case base starts with five past cases stored as a list of Python dictionaries and saved to a JSON file. Each case has a problem part (disaster type, area type, wind speed, rainfall, flood risk, affected population, and warning lead time), a solution part (evacuation strategy, number of evacuees, evacuation centers, shelter capacity, food packs, rescue teams, rescue boats, and medical teams), and an outcome part (a 1–5 success rating with notes). The values are simplified sample data loosely based on real Philippine events, covering typhoons, an urban flood, and an earthquake so that the similarity measure has both close and distant cases to compare.

ID	Past case	Type	Area	Flood risk	Population	Outcome
1	Typhoon Ursula (2019)	typhoon	rural	medium	120,000	4/5
2	Typhoon Odette (2021)	typhoon	urban	high	300,000	3/5 (overcrowded)
3	Tropical Storm Ondoy (2009)	flood	urban	high	500,000	2/5
4	Typhoon Ulysses (2020)	typhoon	urban	high	400,000	3/5
5	Bohol Earthquake (2013)	earthquake	rural	low	200,000	4/5

Explanation of the Similarity Method

We used weighted scoring (nearest-neighbor retrieval). For each feature the program first computes a local similarity from 0 to 1. Disaster type is 1 for an exact match and 0.6 for typhoon vs. flood since typhoons usually bring floods; area type is 1 if it matches and 0 if not; flood risk is treated as ordered levels, so one level apart gives 0.5. Numeric features use 1 − |new − old| ÷ range, which means closer values get a higher score. The global similarity is the weighted average of these local scores, with weights of 2.5 for disaster type, 2.0 for flood risk, 1.5 for area type and population, 1.0 for rainfall and wind speed, and 0.5 for warning time. All past cases are ranked, and the top case is retrieved. For the demo event (urban typhoon with high flood risk), Typhoon Ulysses scored 97.57% and Typhoon Odette 95.93%, while the Bohol Earthquake scored only 24.27%.

Formula: $\text{Similarity}(N, C) = \Sigma \left(w_i \times \text{sim}_i(N_i, C_i) \right) \div \Sigma\, w_i$

Adaptation Logic

The solution of the best case is not copied directly; it is adjusted with six rules. First, the evacuation rate of the old case is kept but multiplied by a risk factor if flood risk changed and by 1.10 if the new rainfall is higher, then applied to the new population. Second, shelter capacity is evacuees plus a 10% buffer, raised to 20% if the past case reported overcrowding, and the number of centers is based on the old case's people-per-center. Third, food packs are three days' worth per evacuee, while rescue and medical teams and boats scale with the evacuee ratio. Fourth, urban

areas with high flood risk get 25% more rescue boats. Fifth, a warning time under 24 hours switches the plan to immediate forced and vertical evacuation, while 48 hours or more allows staged departures. Lastly, a revise check makes sure shelter capacity covers all evacuees before the new case is retained in the case base.

Screenshots of Program Execution

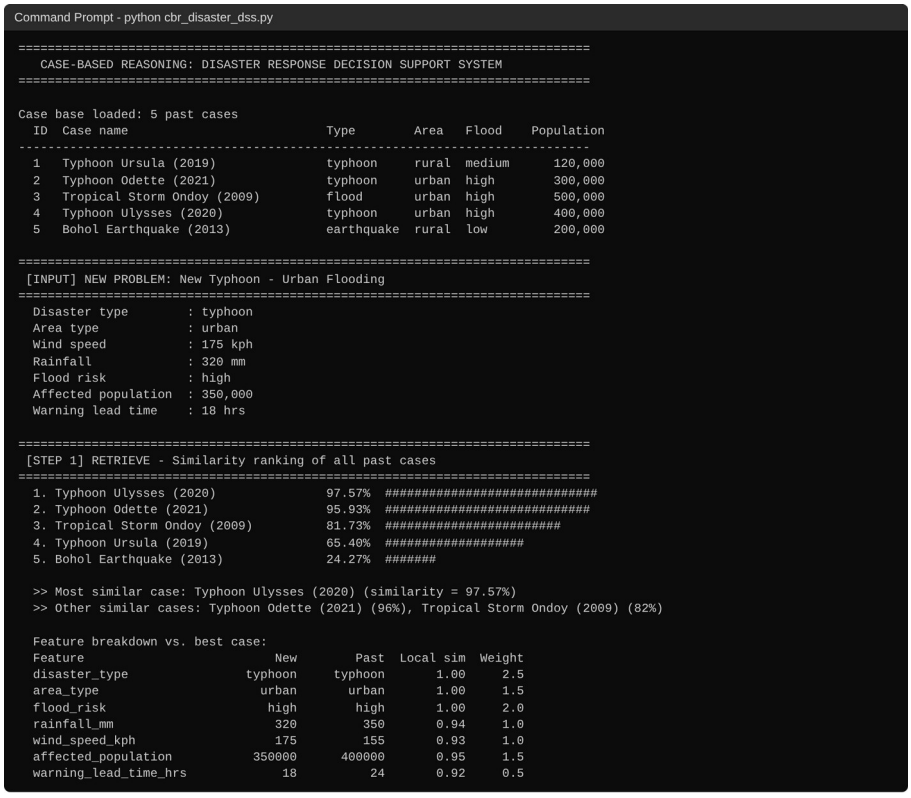


Figure 1. Demo run — case base, new problem (urban typhoon), and similarity ranking (Retrieve)

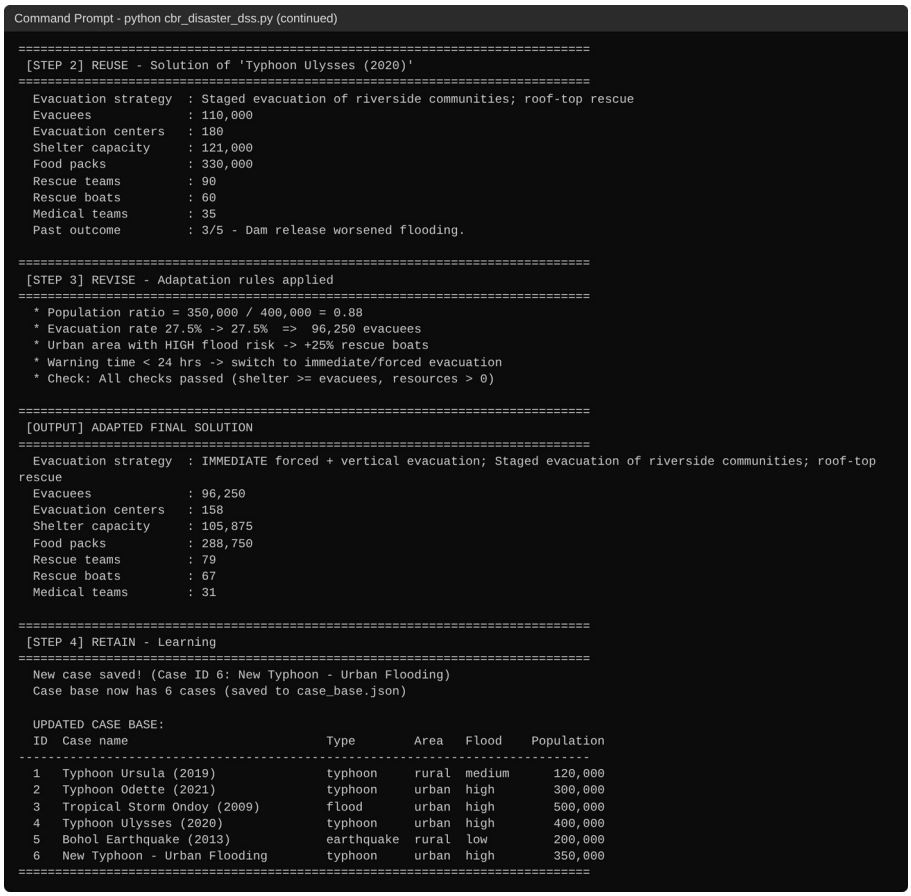


Figure 2. Demo run — Reuse, Revise (adaptation rules), final solution, and Retain with updated case base

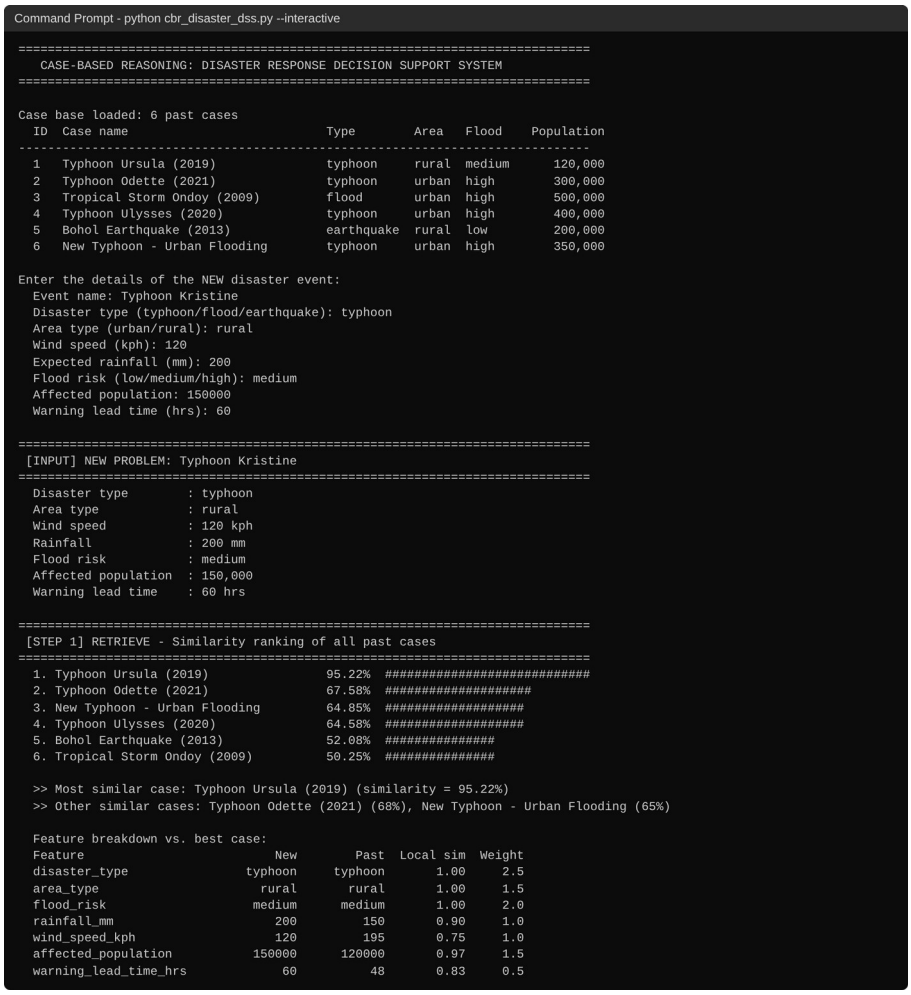


Figure 3. Interactive run — user enters a new rural typhoon; Typhoon Ursula is retrieved

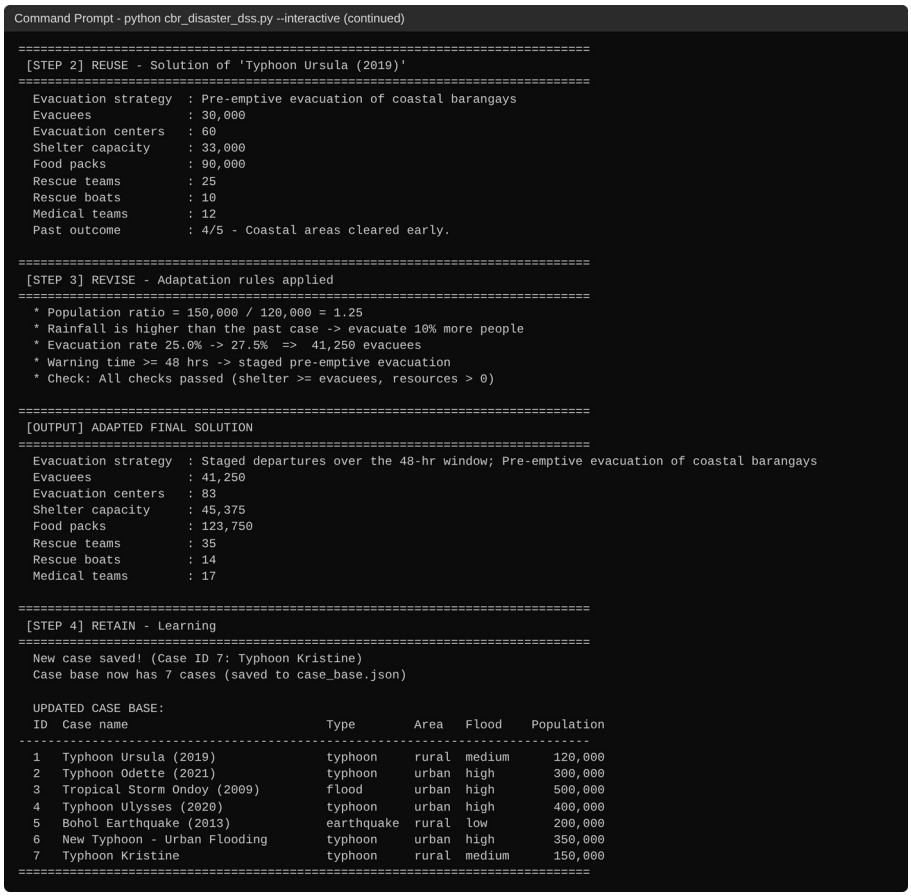


Figure 4. Interactive run — adapted plan and case base updated to 7 cases (learning)

C. Pair Reflection

For this activity, Sangkula focused on the case representation, the similarity function, and the JSON case base, while Sagpang handled the adaptation rules, testing, and the documentation and screenshots, though we debugged the program together. Our biggest challenge was choosing fair weights so that related typhoon and flood cases ranked high while unrelated disasters like earthquakes ranked low, which is why disaster type and flood risk got the biggest weights. We also had trouble with rounding, where small decimal errors made the evacuee count go up by one, which we fixed with a helper function. Through this activity, we learned that CBR does not need a complete set of rules because it solves new problems by reusing and adjusting past experience, which is how LGUs actually plan using previous typhoons. We also saw how the retain step lets the system improve, since every new case we saved became a possible match for the next event.